OptSem-C: Public Optimizer Contracts for Query Engine Portability

Haoyi Zhang
Xi'an Jiaotong-Liverpool University
Suzhou, China
hyeliozhang@gmail.com

Huaijin Ran
Nanyang Technological University
Singapore, Singapore
huaijin003@e.ntu.edu.sg

ABSTRACT

Optimizer-behavior comparisons can fail before a performance number is measured: the comparison denominator is often a coarse vocabulary rather than an exact public contract. Engine-selection, migration, and performance-debugging studies routinely compare labels such as `pushdown`, `join`, or `cost-based`. Such labels can collapse local rewrites, source delegation, service-side stages, and runtime decisions into the same apparent capability, causing a study to compare different query-processing phenomena.

This paper presents OptSem-C, a benchmark methodology for public optimizer behavior contracts. A contract rule is a guarded modal statement over query features with explicit operator, action variant, optimizer layer, execution placement, decision time, observability, version, and public evidence. The semantics separates behavioral worlds from evidential support, defines a conflict-aware state join, and treats lossy comparison as a projection kernel over exact contract signatures. OptSem-C therefore lets a study publish the vocabulary it preserves, the equivalences it assumes, and the repair fields needed to make those assumptions safe.

The grounded study contains 287 source-linked rules and 4,216 SQL probes over 7,112 valid optimizer-feature interactions. Keyword projection declares 2,284 engine–probe equivalences; 254 are false under exact public contracts, while the remaining 2,030 agree under the exact signature schema. Operator-only projection declares 2,268 equivalences, including 238 projection-induced contract collisions. Published-motif coverage maps optimizer and workload families from prior studies to 90 executable requirements; two-engine checks validate the generated SQL and visible-plan layers. Exhaustively evaluating all 256 retained-field vocabularies shows that 44 remain unsafe; after action-variant identifiers are removed, the minimum safe semantic vocabularies have size two. Across the three headline projections (keyword, operator-only, and yes/no), layer+placement separates all 498 finite-corpus witnesses. OptSem-C turns optimizer-contract comparison from an informal feature checklist into a finite, auditable query-processing relation.

PVLDB Reference Format:
Haoyi Zhang and Huaijin Ran. OptSem-C: Public Optimizer Contracts for Query Engine Portability. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Availability Statement:
The source code, data, and/or other artifacts have been made available at https://github.com/Haoyi-Zhang/OptSemC.

1 INTRODUCTION

SQL deliberately hides physical execution choices. That abstraction is useful for applications, but it is dangerous for optimizer comparison. The same query can be accepted by several analytical engines while exposing different public behavior: a local predicate rewrite in one engine, connector-side execution in another, a service-side stage graph in a third, and only a logical explanation in a fourth. These differences determine what a benchmark author, system integrator, or database researcher may conclude about join ordering, pushdown, materialization, adaptive execution, and plan evidence before any deployment-specific performance experiment begins.

The practical failure mode is not cosmetic. A migration checklist may equate local filter rewrite with remote-source delegation; an engine-selection study may compare compile-time join enumeration with runtime repartitioning; a debugging report may treat an estimated plan node and a service profile stage as the same evidence. OptSem-C tells such studies which public optimizer distinction they preserved and which repair fields make the comparison auditable.

Database research has precise abstractions for relational semantics, optimizer search, cardinality estimation, adaptive execution, and query equivalence [11, 18, 35–37, 47, 66, 69]. Public optimizer behavior is usually compared with much coarser vocabularies. A cell labeled "pushdown" may mean predicate movement inside the engine, partition pruning, aggregation delegation, join delegation, limit delegation, or runtime filtering. A cell labeled "join" may refer to a logical operator, an estimated physical operator, a distributed exchange, or a runtime profile entry. A claim that two engines are "cost based" may hide whether statistics are local, connector-provided, unavailable, or revised during execution. These shortcuts are not harmless summarization: they can create projection-induced contract collisions, where a coarse vocabulary declares equality among public contracts whose exact signatures differ.

OptSem-C treats this failure as a query-processing systems problem with formal foundations. A *public optimizer contract* is the optimizer behavior an engine exposes through public plan interfaces, tuning surfaces, and system descriptions. Documentation can support a contract that is not visible in a particular textual plan, and a plan tag can expose behavior without proving the private cost model. The object is therefore not a hidden implementation trace and not a latency measurement. OptSem-C represents each public statement as a guarded modal rule over query features. The rule records the action, the guard under which it applies, where the action is placed, which optimizer layer owns it, when the decision

is made, how the behavior is observable, and whether the public contract requires, permits, forbids, or does not specify the action.

Research questions. We ask three finite, auditable questions. RQ1 asks whether public optimizer claims can be encoded as source-linked contracts and executable probes without becoming a latency benchmark. RQ2 asks how often comparison vocabularies that appear in public feature claims collapse distinct source-supported contracts. RQ3 asks which semantic fields eliminate those collisions, and whether the repair survives held-out sources, query-feature families, and engine-pair families.

The central result is a projection-kernel account of these contract collisions. Comparing optimizer contracts through a keyword or operator vocabulary applies a predeclared projection to exact signatures. If exact signatures disagree but fall into the same projected class, the projection has declared an equivalence that the public record refutes. This is a statement about the comparison denominator, not a claim that every witness implies a latency regression or private implementation mismatch. OptSem-C fixes the contract schema before the projection audit, computes exact and coarse classes from the same source-linked records, and reports finite repairs as certificates. Keyword projection declares 2,284 engine–probe equivalences: 2,030 have matching exact signatures and 254 are refuted. Operator-only projection declares 2,268 equivalences, including 238 collisions. Restoring placement repairs keyword collisions; restoring optimizer layer repairs operator-only collisions; restoring layer and placement repairs all 498 headline witnesses. A field-only stress test over all 256 retained-field vocabularies leaves only 44 unsafe regions, all covered by concrete counter-witnesses; after excluding action-variant identifiers, no singleton semantic field is safe.

We also introduce OptSemBench-C, a covering benchmark for optimizer-contract behavior. Its denominator is not raw query count. It is coverage of optimizer-relevant feature interactions: source boundary, connector capability, join shape, predicate class, aggregation, statistics need, reuse, distribution, adaptivity, and control surface. The generated probes are executable SQL representatives, and the probe space is cross-checked against optimizer-workload motifs from join-order studies, exact-cardinality optimizer testing, cardinality-estimation benchmarks, adaptive query processing, data-lake and federated workloads, star-schema analytics, and optimizer-control surfaces [4, 12, 24, 50, 54, 55, 58, 60]. We use this audit as a published-motif vocabulary stress test: it asks whether comparison words used around published workloads have concrete semantic requirements in our probe space, not whether the original studies made the same claim or measured the same performance outcome.

Contributions. (1) We define public optimizer behavior contracts as a query-processing object distinct from SQL result equivalence and runtime performance. (2) We give a guarded modal representation with evidence linking and conflict-aware state join for cross-engine behavior comparison. (3) We give a finite projection calculus and repair-certificate workflow for publishing the assumptions of an optimizer-comparison vocabulary. (4) We introduce OptSemBench-C, a covering query-probe benchmark linked to published workload motifs, and provide a source-linked corpus across seven analytical SQL engines with executable SQL and visible-plan checks. (5) We show that coarse optimizer comparisons induce contract-level collisions across a broad projection portfolio, and that the missing fields are precise query-processing semantics rather than lexical synonyms.

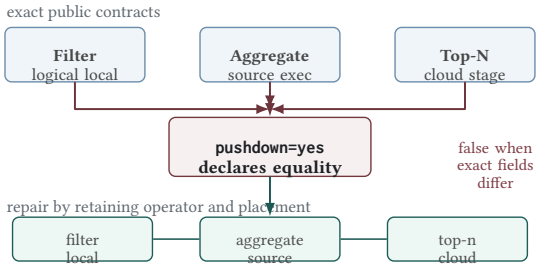

Figure 1: Projection loss in optimizer comparison. Exact contracts collapse into one coarse `pushdown=yes` cell; retaining operator and placement separates the false equality.

2 THE FAILURE MODE

Figure 1 gives the smallest example. A feature matrix maps several public optimizer contracts into the same label. After the mapping, behaviors with different placement, action kind, and evidence layer become indistinguishable, so a study that treats the label as equality can compare non-equivalent public optimizer behavior.

Feature-name collision. A cell such as `pushdown=yes` may denote predicate movement inside an engine, partition pruning, connector-side predicate delegation, aggregation delegation, join delegation, limit pushdown, Top-N delegation, or runtime filtering. These actions affect different operators and have different guards, placements, decision times, and observable plan effects. Collapsing them loses the distinctions optimizers use to choose access paths, plan operators, distribution strategies, and reoptimization points [2, 5, 19, 34, 48, 59, 61, 68].

Operator disappearance. A missing native plan node is not a reliable semantics. It may indicate source delegation, logical-plan observability, rewrite, stage-level aggregation, or runtime-only visibility. OptSem-C represents source-delegated execution as a positive contract action rather than as absence. This matters for modern analytical systems whose public surfaces combine local plans, cloud-service stages, connector adapters, and logical explanation outputs [7, 22, 41, 64, 67, 75].

Architecture ranking. A local engine should not be ranked below a distributed engine merely because broadcast, shuffle, or remote-source delegation is not applicable. Conversely, a federated or cloud engine should not be credited with local optimizer behavior when the public contract exposes delegation or service-side stages. OptSem-C separates non-applicability, documented absence, documented optional behavior, and unspecified evidence.

Observable contract versus hidden implementation. The object of study is not what an engine could do internally. It is what the public contract supports as a comparison claim. This distinction is analogous to the separation between SQL equivalence and optimizer-contract equivalence: result-preserving rewrites can be

semantically equivalent while exposing different optimizer behavior, different evidence layers, and different portability obligations [1, 10, 15, 16, 71].

These failures imply a minimum payload for comparing public optimizer behavior: guard, canonical action, modality, placement, decision time, observability, version, and provenance. The rest of the paper formalizes this payload and measures what happens when a comparison vocabulary projects it away.

3 CONTRACT SEMANTICS

A query probe is a SQL skeleton paired with a finite feature vector and validity constraints. Features describe optimizer-relevant structure: join shape and type, predicate class, aggregation, order/limit behavior, source boundary, connector capability, statistics need, reuse structure, distribution trigger, adaptivity trigger, and optimizer control surface. They are not workload-frequency estimates; they are coordinates on the public optimizer decision surface.

Definition 1 (Public contract rule). *Let $\mathcal{A}$ be the finite domain of canonical optimizer actions. An action records operator class, action kind, optimizer layer, execution placement, decision time, observability, and variant. The state set $\mathbb{S}$ contains* MUST, MAY, MUST_NOT, *and* UNSPEC. *A public contract rule is $r = (\gamma, a, s, \nu, \epsilon)$, where γ is a guard over query features, $a \in \mathcal{A}$, $s \in \mathbb{S}$, ν is the version or time window, and ϵ is evidence provenance. For engine e, C_e is the finite rule set, and $C_e(q) = \{r \in C_e \mid \gamma_r(\phi(q)) = \text{true}\}$ is the rule set applicable to probe q.*

The state domain has two components that feature matrices usually conflate. Behaviorally, MUST requires an action, MUST_NOT forbids it, and both MAY and UNSPEC leave it possible. Evidentially, MAY is public support for optional behavior, while UNSPEC is lack of public support. Equivalently, a state is interpreted as (m, u, e): required behavior, allowed behavior, and evidence support. This is why documented optional pushdown and undocumented implementation possibility are different public contracts.

For each engine-probe pair, OptSem-C constructs a contract map $M_{e,q} : \mathcal{A} \to \mathbb{S}$, defaulting every action to UNSPEC. Applicable rules update the map with a conflict-aware state join $\sqcup$. The join strengthens unspecified actions when public evidence is found, preserves compatible stronger evidence, and reports contradictions instead of averaging them. The evidence component follows database-provenance practice: support is explicit, inspectable, and attached to the object it justifies [8, 13, 38, 39].

Table 1: Conflict-aware state join for contract support states (U=UNSPEC, N=MUST_NOT, C=CONFLICT).

$\sqcup$	U	MAY	MUST	N
U	U	MAY	MUST	N
MAY	MAY	MAY	MUST	C
MUST	MUST	MUST	MUST	C
N	N	C	C	N

Definition 2 (State-join algebra). *The binary operator $\sqcup$ is the partial join shown in Table 1. It is total over non-conflicting pairs and returns* CONFLICT *for pairs that simultaneously require and forbid, or support and forbid, the same canonical action. Let $x \preceq y$ denote that $x \sqcup y = y$ for non-conflicting states. Then* UNSPEC *is the least informative state, while* MUST *and* MUST_NOT *are incomparable incompatible commitments.*

Lemma 1 (Merge invariants). *On non-conflicting states, $\sqcup$ is commutative, associative, idempotent, and monotone with respect to $\preceq$. Therefore a contract map obtained by joining applicable rules is independent of rule order.*

Proof. The properties follow by inspection of the finite table. Idempotence holds because every diagonal entry is the input state. Commutativity holds because the table is symmetric. Associativity and monotonicity are checked over the finite non-conflicting triples; the only excluded triples are exactly those containing incompatible positive and negative commitments. Lifting the pointwise operator to maps preserves the properties for every action independently. □

Let $W \subseteq \mathcal{A}$ be an abstract optimizer-behavior world. A map M denotes the world set $[\![M]\!] = \{W \mid M(a) = \text{MUST} \Rightarrow a \in W,\ M(a) = \text{MUST\_NOT} \Rightarrow a \notin W\}$. The evidential support set is $\text{Supp}(M) = \{a \in \mathcal{A} \mid M(a) \in \{\text{MUST}, \text{MAY}, \text{MUST\_NOT}\}\}$. Behavioral containment and evidential support are intentionally separate: two maps can allow the same worlds while differing on whether an action is publicly documented.

Definition 3 (Exact signatures, kernels, and projections). *The exact evidence-supported signature of M is $S(M) = \{(a, M(a)) \mid a \in \text{Supp}(M)\}$. Exact equivalence is equality of signatures over full evidence atoms. A projection is a declared map $\pi : \mathcal{A} \times \mathbb{S} \to \mathcal{B}$ from full evidence atoms to a comparison vocabulary such as keyword, yes/no, or operator-only labels. The projected signature is the finite image $S_\pi(M) = \{\pi(a, M(a)) \mid a \in \text{Supp}(M)\}$. The projection kernel is the equivalence relation $\mathcal{K}_\pi$ where $M_i \mathcal{K}_\pi M_j$ iff $S_\pi(M_i) = S_\pi(M_j)$. A projection-induced contract collision is a pair in $\mathcal{K}_\pi$ whose exact signatures differ.*

For quantitative comparison, exact compatibility is the Jaccard overlap $|S(M_i) \cap S(M_j)|/|S(M_i) \cup S(M_j)|$, and projected compatibility applies the same formula to S_π. The denominator is evidence-supported: undocumented actions do not count as evidence of either equality or inequality.

Theorem 1 (Finite evaluation). *For finite feature and action domains, constructing $M_{e,q}$ from finite C_e and q is decidable.*

Proof. Every guard is a finite Boolean formula over a finite feature vector. The applicable rule set is finite, and every applicable rule performs one lookup in the finite state-join table. Construction therefore terminates. □

Theorem 2 (Projection-kernel collision). *For any projection π, a projection-induced contract collision is exactly the nontrivial part of the kernel $\mathcal{K}_\pi$: M_i and M_j collide iff $S(M_i) \neq S(M_j)$ and $M_i \mathcal{K}_\pi M_j$. If π is non-injective on evidence atoms, there exist contracts with such a collision witness.*

Proof. The first statement unfolds the definitions: $\mathcal{K}_\pi$ is equality after projection, while exact non-equivalence is $S(M_i) \neq S(M_j)$. For existence, take distinct evidence atoms $(a_1, s_1) \neq (a_2, s_2)$ with

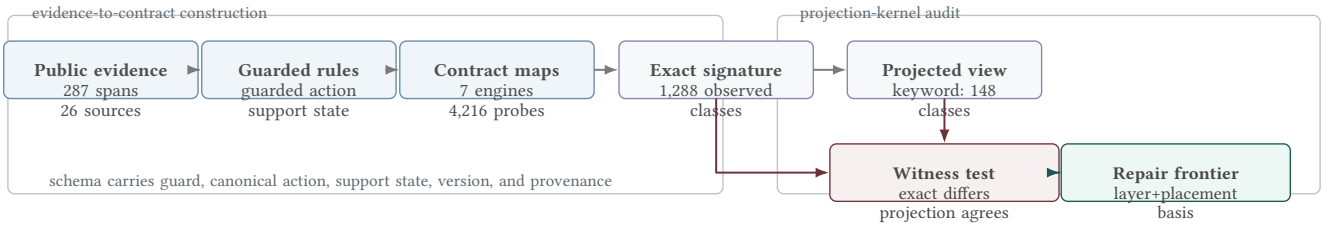

Figure 2: Contract-comparison pipeline. OptSem-C converts public evidence into exact engine-probe signatures, tests projected equality with witnesses, and reports repair fields for collision kernels.

$\pi(a_1, s_1) = \pi(a_2, s_2)$. Let M_1 contain only (a_1, s_1) and M_2 contain only (a_2, s_2). Their exact signatures differ, but their projected signatures are the same singleton. □

Definition 4 (Projection information profile). *For a finite set of observed maps Y, let $H(S)$ be the empirical Shannon entropy of exact signatures and $H(S_\pi)$ the entropy of projected signatures. The entropy-retention ratio $\rho_\pi = H(S_\pi)/H(S)$ measures distinguishability retained by the comparison vocabulary. The projected collision count is the number of projected-signature classes containing more than one exact signature class.*

Lemma 2 (Kernel inflation). *If a projection declares E_π equivalent pairs and exact comparison declares E over the same pair set, then E_π/E is the pairwise inflation factor induced by the kernel. Every collision witness contributes to this inflation, and a zero-collision strengthened projection must have $E_\pi = E$.*

Proof. Exact equivalence implies projected equivalence only when the projection is a function of the exact atom payload, so $E_\pi \geq E$. The additional pairs counted by E_π are exactly the unequal exact signatures that collapse under π, which are collision witnesses. If there are none, no additional pairs are counted and the two denominators agree. □

Theorem 3 (Evidence refinement). *Adding a non-conflicting evidence-backed rule cannot silently weaken a public contract: it behaviorally refines possible worlds, evidentially refines support, or both.*

Proof. A MUST or MUST_NOT rule removes worlds from $[\![M]\!]$. A MAY rule may leave worlds unchanged, but changes an action from UNSPEC to evidence-supported. Because the rule is non-conflicting, the state join does not erase existing support or replace a compatible stronger state with a weaker one. □

The remaining formal statements are a projection calculus for finite comparison audits. They are not meant to make information loss surprising; they make the certificate obligations explicit, so Section 5 can measure which query-processing fields are empirically sufficient. Let F be the semantic-field universe and let π_R be the repaired projection that appends the field values in $R \subseteq F$ to $\pi(a, s)$ for each evidence atom (a, s). For a finite grounded comparison set X, define $X_{\pi_R} = \{(M_i, M_j) \in X \mid S(M_i) \neq S(M_j) \wedge M_i \mathcal{K}_{\pi_R} M_j\}$. A field set R is safe, or universal, for π on X when $X_{\pi_R} = \emptyset$; otherwise it is unsafe and has a grounded counter-witness.

Theorem 4 (Projection-lattice frontier). *If $R \subseteq R' \subseteq F$, then $X_{\pi_{R'}} \subseteq X_{\pi_R}$. Consequently the safe field sets are upward closed, the unsafe field sets are downward closed, and the minimum universal repairs form an antichain frontier in the finite field lattice.*

Proof. Projection $\pi_{R'}$ emits every component emitted by π_R plus additional field values. Adding fields can split projected equivalence classes, but cannot merge classes already separated. Therefore any unequal pair still collapsed after adding R' was also collapsed after adding R. Upward closure of safe sets and downward closure of unsafe sets follow immediately, and minimal safe sets are incomparable by set inclusion. □

For a witness $w = (M_i, M_j)$, define its separator family $\mathcal{D}_w = \{R \subseteq F \mid M_i \not\mathcal{K}_{\pi_R} M_j\}$. By Theorem 4, each $\mathcal{D}_w$ is upward closed in the field lattice, and universal repair is the finite intersection problem $R \in \bigcap_{w \in X_\pi} \mathcal{D}_w$.

Theorem 5 (Repair certificate). *Fix a projection π and finite comparison set X. A field set R eliminates all observed collisions iff R belongs to every witness separator family $\mathcal{D}_w$ for $w \in X_\pi$.*

Proof. If $R \in \mathcal{D}_w$ for every witness, every pair collapsed by π becomes separated under π_R, so $X_{\pi_R} = \emptyset$. Conversely, if $X_{\pi_R} = \emptyset$, every original witness is separated under π_R, which is exactly membership in each separator family. □

Theorem 6 (Resolution-lattice certificate). *For a finite comparison set X and finite field universe F, exhaustive enumeration of all retained-field projections over $2^{|F|}$ subsets is a complete certificate of which public optimizer fields are sufficient for safe comparison on X. If a subset R is unsafe, a single pair $(M_i, M_j) \in X$ with $S(M_i) \neq S(M_j)$ and $S_R(M_i) = S_R(M_j)$ is a checkable counter-certificate. If R is safe, direct equality testing over all pairs in X is a checkable certificate of safety.*

Proof. For each subset $R \subseteq F$, the retained-field signature S_R is a deterministic projection of the exact signature. The comparison set X is finite, so checking all pairs in X terminates. Unsafety is existential and is witnessed by one collapsed unequal pair. Safety is universal over the same finite set and is witnessed by the absence of such a pair after exhaustive enumeration. Since every subset of F is enumerated, the resulting safe and unsafe regions are complete for the chosen field universe. □

Theorem 7 (Minimum repair as hitting set). *For finite semantic fields F and collision witnesses X_π, deciding whether there is a*

universal repair of size at most k is NP-complete in $|F| + |X_\pi|$. For the fixed OptSem-C field universe, every minimum repair is exactly computable, and a certificate consists of the repaired witnesses plus counter-witnesses for every smaller field set.

Proof. Membership in NP follows by checking a candidate field set against every finite witness. For hardness, reduce Hitting Set. Given sets $H_1, \ldots, H_m \subseteq F$, create one collision witness per H_i whose two contracts agree under the baseline projection and differ exactly on the fields in H_i. A field set repairs that witness iff it intersects H_i; hence a universal repair of size k exists iff the hitting-set instance has a hitting set of size k. Since OptSem-C fixes a finite field universe, enumerating field subsets in increasing cardinality returns all minimum repairs. Exhausting all smaller subsets gives the lower-bound part of the certificate. □

Theorem 8 (Semantic repair basis). *Fix a finite comparison set X, projections Π, and a semantic field set F that excludes action identifiers such as fine-grained variants. If $R \subseteq F$ is universal for every $\pi \in \Pi$, then augmenting each projection with R eliminates all observed collisions without relying on action-identifier labels.*

Proof. Apply Theorem 5 to each projection independently. Because X and Π are finite, the union of their witness sets is finite; a common R that lies in every corresponding separator family leaves no collapsed unequal pair. Excluding action identifiers restricts the admissible field universe but does not change the certificate condition. □

The semantics makes the comparison payload explicit. A user comparing only operator names chooses a projection that drops placement, decision time, variant, and observability. A feature checklist chooses an even coarser projection that can drop operator class itself. OptSem-C does not forbid those projections; it computes exactly which kernel collisions they induce, which grounded counter-witnesses remain, and which query-processing fields repair the comparison.

4 BENCHMARK AND GROUNDED CORPUS

OptSemBench-C is a behavior-contract benchmark. It does not measure latency or rank engines. It generates probes that cover optimizer-relevant feature interactions and then measures which public contracts those probes activate. This follows the benchmark principle that the metric and denominator must match the object of study: plan-quality benchmarks ask whether cardinality estimates improve plans, data-lake benchmarks expose explicit discovery denominators, and optimizer-contract benchmarks must expose semantic traps rather than only query counts [14, 42, 43, 51, 56, 57, 72, 74].

Let $F = F_1 \times \cdots \times F_n$ be the finite feature domain and Ψ the validity constraints. A target interaction is a valid partial assignment over a feature subset. The schema and feature domain are fixed before witness counting and derive from standard query-processing constructs and cited public optimizer surfaces, not from witnesses or motifs. The benchmark targets global pairwise coverage plus selected three-way interactions for source boundary, connector capability, operator class, statistics need, join shape, adaptivity trigger, and distribution trigger. A greedy generator selects renderable

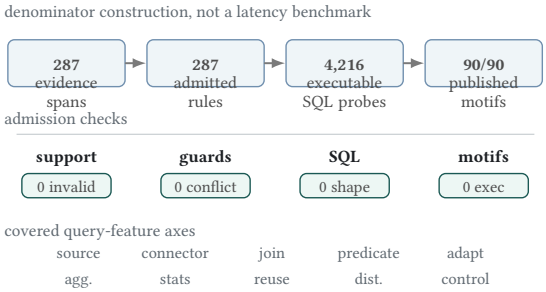

Figure 3: Corpus, benchmark, and execution denominators. The spine links countable source evidence, generated SQL probes, admission checks, and published-motif coverage.

full assignments until all renderable target interactions are covered. The generator has two responsibilities. First, it separates benchmark adequacy from any particular system: adequacy is measured against the feature-interaction universe, not against runtime results. Second, it produces probes that are diagnostic rather than workload-frequency estimates. A probe is useful when it activates a contract guard or separates two contract signatures.

Theorem 9 (Coverage). *If every valid target interaction has at least one renderable full assignment, the generator terminates with a probe set covering every renderable target interaction.*

Proof. The target universe is finite. Each selected assignment covers at least one uncovered interaction. Thus the number of uncovered interactions decreases monotonically until it reaches zero. □

The main empirical corpus contains only source-linked grounded rules. Each rule records a public evidence span, source URL, line range, digest, and source class. Claims without grounded evidence, expressible guards, or public observability are outside the finite denominator rather than negative evidence. Thus the paper claims coverage of the declared public-contract denominator, not a census of private optimizer rules. The corpus covers plan observability, pushdown, join search, distribution, materialization, runtime adaptivity, statistics, and optimizer control surfaces. Every grounded rule is triggered by at least one generated probe, and contract merge produces no conflicts.

The source set is fixed before witness counting and includes public references for all engines in the study. Examples include PostgreSQL planning documentation, Trino cost-based optimization and pushdown documentation, BigQuery stage and query-plan documentation, Spark SQL adaptive-execution documentation, Snowflake logical-plan documentation, DuckDB CTE materialization documentation, and ClickHouse join documentation [17, 30, 31, 33, 40, 45, 46]. These sources are not used as performance measurements. They provide the public contracts that the framework formalizes.

The grounded corpus is intentionally conservative. A rule is admitted only when the evidence span supports an optimizer action, a guard or applicability condition, and an observability or placement interpretation. Vague performance advice is excluded from the main

Table 2: Representative source-grounded contract rules with evidence-line spans and paraphrases.

Engine	State	Action	Lines	Evidence paraphrase
PostgreSQL	MAY	Plan/observe/local	L46–L56	EXPLAIN exposes estimated startup and total cost, output rows, row width, and planner-cost units.
BigQuery	MAY	Exchange/adapt/cloud	L990–L992	The query plan may be modified during execution by adding repartition stages.
Snowflake	MAY	Plan/observe/cloud	L150–L154	EXPLAIN compiles a statement and returns a logical execution plan rather than executing it.
ClickHouse	MAY	Join/adapt/local	L313	With automatic join-algorithm selection, execution can switch from hash join to partial merge join.
DuckDB	MAY	Materialize/inline/local	L482–L487	CTE inlining is guarded by single use, absence of volatility, no grouped aggregation, and inlining hints.
Spark SQL	MAY	Plan/adapt/distributed	L129–L133	Adaptive query execution uses runtime statistics and can re-optimize a query while it is running.
Trino	MUST_NOT	Agg./pushdown/source	L168–L176	Aggregation pushdown does not support aggregate calls containing expressions.

corpus. This policy reduces corpus size but strengthens the central claim: the evaluation is about contracts that are publicly supported, not about inferred implementation behavior.

Contract extraction and verification. Each admitted rule records the engine, version scope, source identifier, digest, action fields, guard, and observability surface. Human coding maps public evidence spans into the fixed schema; scripts check schema validity, source linkage, guard reachability, trigger coverage, merge conflicts, SQL executability, and table-source consistency. The field schema and projection portfolio are replay inputs; kernel counts, repair fields, and tables are regenerated from them, so the repair basis is not chosen after inspecting one highlighted collision family. Documentation admits the public claim; EXPLAIN, profile, or stage surfaces bind exposed plan evidence when available; SQL execution validates the probe denominator rather than private optimizer truth. A counted witness requires two source-supported exact signatures, projection equality under the declared vocabulary, and separation by the reported repair field.

The denominator dashboard in Table 3 checks that this conservatism does not create unreachable contracts or syntactic probes. A finite-disjunction guard semantics evaluates each admitted rule against the generated probe denominator. All 287 grounded rules have at least one triggering probe; 252 depend on query features rather than global engine labels; and no guard uses a feature dimension or value outside the benchmark domain.

Coverage is reported over two denominators. The first is benchmark coverage: every targeted renderable valid feature interaction is covered by at least one generated probe. The second is contract triggering: every grounded rule is applicable to at least one probe. Together these denominators make the empirical scope explicit. A rule that cannot be triggered by any probe would indicate a benchmark gap; a feature interaction that cannot be rendered is excluded from the coverage denominator and reported as a modeling constraint.

The benchmark also exposes executable SQL shapes. Each generated probe has a normalized SQL skeleton and shape checks for joins, filters, grouping, ordering, limits, and reuse structure. The full 4,216-probe corpus is accompanied by a compact 48-probe motif-cover prefix. The real-engine motif execution set uses the deterministic union of this motif cover and residual feature-axis maxima; it is not a hand-picked success set, and it is not a replacement for the full evaluation.

A common failure mode in generated benchmarks is to produce feature-complete but non-executable query text. OptSemBench-C therefore validates every SQL skeleton on a deterministic relational catalog with local, remote, and cross-source namespaces. This validation is not a performance baseline and is not exhaustive contract-trigger proof. Its role is stricter and more basic: every generated probe must be parseable, plannable, executable, and shape-consistent with its feature vector. Table 3 reports the validation result: all 4,216 generated probes obtain a plan and execute, with zero SQL-shape failures. Repeating the full corpus across three deterministic catalog populations yields 12,648 successful probe-catalog executions, and production-engine checks on DuckDB and PostgreSQL add 8,432 full-corpus engine–probe executions plus 142 motif-representative executions with zero execution failures.

Contract validation is layered. Guard reachability says that each public rule has a generated probe satisfying its admitted guard; SQL execution says that the denominator is not syntactic fiction. Visible-plan sanity checks on DuckDB and PostgreSQL expose expected plan tokens at mean rates 0.705 and 0.796 over the full corpus, and 0.756 and 0.819 on motif representatives. These checks do not reverse-engineer private optimizers or prove every documented behavior fires on every cost-model instance. A counted witness still requires source-supported exact signatures, guard reachability, projection equality, and repair separation; runtime plan satisfaction is a separate engine-specific validation layer.

The same fixed probes support two budgeted evaluation orders. The cover order is the generator's feature-interaction order. The diagnostic order greedily maximizes newly triggered grounded rules without adding any new probe. Table 4 reports both. The cover order triggers all grounded rules within 100 probes; the diagnostic order triggers 286 of 287 rules within 50 probes and all rules within 100. Random subsets of 100 probes trigger 81% of rules on average and 85% at the 95th percentile. Thus OptSemBench-C is not merely large enough to cover the surface; it also contains compact diagnostic prefixes that expose almost all grounded contracts.

A benchmark for public optimizer contracts must connect to existing database workloads without becoming another latency benchmark. We therefore add a crosswalk from published optimizer and benchmark motifs to OptSemBench-C feature requirements: decision-support structure, JOB join-order sensitivity, SSB star schemas, cardinality-estimation plan quality, adaptive replanning, cross-source/data-lake joinability, optimizer controls, join

Table 3: Benchmark denominator and validation checks across corpus, SQL, execution, and motif layers.

Layer	Check	Value	Scope	Interpretation
Corpus	grounded rules	**287**	26 sources	finite public-contract relation
Corpus	probe-supported rules	**287/287**	median support 102	every admitted rule is triggerable
Corpus	non-global guards	252	3 containments	query-conditional, audited overlaps
Guard schema	invalid dimensions	**0**	fixed feature domain	no out-of-domain guard values
SQL corpus	generated probes	**4,216**	digest `df107f018e37`	full executable denominator
SQL corpus	motif-cover prefix	48	human inspection	compact replay slice, not cherry-picking
Execution	planned/executed probes	**4,216/4,216**	91 distinct plans	every generated probe plans and runs
Execution	shape/exec failures	**0/0**	18,448 rows	validation checks find no SQL-shape or runtime failures
Published motifs	covered requirements	**90/90**	12 families	every declared motif has executable probe support

Table 4: Probe budgets for triggering grounded contract rules; random columns summarize subset baselines.

Budget	Cover	Diagnostic	Random mean	Random 5–95
50	.672	.997	.707	[.627,.767]
100	**1.000**	**1.000**	.811	[.770,.850]
250	**1.000**	**1.000**	.890	[.864,.913]
500	**1.000**	**1.000**	.920	[.899,.941]
1000	**1.000**	**1.000**	.951	[.934,.969]
2000	**1.000**	**1.000**	.973	[.958,.986]
4216	**1.000**	**1.000**	**1.000**	**[1.000,1.000]**

algorithms, and Spark/Trino distribution controls [3, 6, 20, 21, 23, 27, 44, 49, 62, 70]. Figure 4 reports the executable denominator: all 90 declared motif requirements across 12 families are covered by the generated probe space. The crosswalk is a coverage audit over optimizer-decision surfaces, not a copy of third-party benchmark SQL or a claim about third-party latency results. Here "published" means that the motif family is named in prior workload or optimizer studies outside OptSem-C; these motifs are checked after the schema is fixed and are not used to select the repair fields.

This crosswalk is also the non-circularity check for the probe denominator. The projection-kernel measurements use the OptSem-C feature vocabulary, but a counted witness must satisfy independent conditions: two public evidence-supported signatures differ, a declared comparison vocabulary collapses them, both sides have source support, and the reported repair separates the pair. The published-motif denominator is not learned from those witnesses; it asks whether third-party optimizer and workload families have concrete representatives in the same probe space. Motif requirements that cannot be represented under the current public feature domain would be reported as denominator misses rather than silently dropped. We therefore claim contract-denominator adequacy and published-motif coverage, not that generated probes reproduce third-party workload performance.

The motif-difficulty view prevents a misleading denominator. Some motifs are broad and match many probes, such as star-schema group-by shapes. Others are sparse, such as a TPC-H conjunctive-filter motif that has one matching rendered probe under the current validity constraints. Reporting both coverage and matching-probe counts makes benchmark adequacy inspectable: coverage says the motif is present, while difficulty says how concentrated or redundant that coverage is. The executable motif suite adds a second denominator check: for each published motif, the system selects one satisfying probe and validates the generated SQL on the deterministic catalog, yielding 90 validated motif representatives and

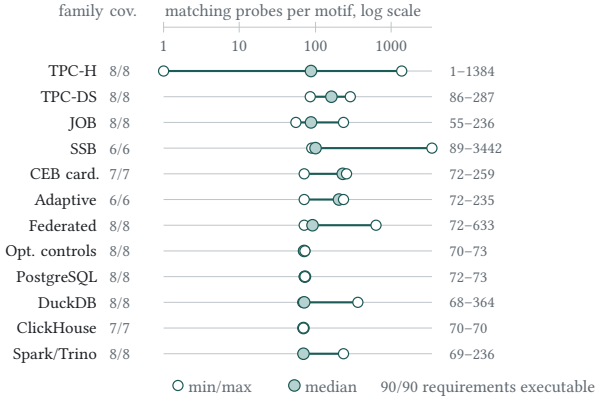

Figure 4: Published optimizer-motif denominator. The log-scale range plot shows min/median/max matching probes per family; all 90 requirements are executable.

Table 5: Matching-probe difficulty across published optimizer motifs; min/median/max count probes per motif.

Suite	Motifs	Min	Median	Max
TPC-H	8	1	87.0	1384
TPC-DS	8	86	164.5	287
JOB	8	55	87.0	236
SSB	6	89	101.0	3442
CardEst	7	72	228.0	259
Adaptive	6	72	205.5	235
Federated	8	72	91.5	633
Controls	8	70	72.0	73
PostgreSQL	8	72	73.0	73
DuckDB	8	68	72.0	364
ClickHouse	7	70	70.0	70
Spark/Trino	8	69	70.0	236

383 result rows. Thus the motif denominator is not a list of names; it is a set of optimizer-decision requirements with concrete SQL representatives. Engine-plan validation is a separate experiment because the motif suite itself is a denominator check rather than a claim that published benchmark queries and generated probes induce identical plans.

5 EVALUATION

The evaluation asks four questions: how often coarse comparisons collapse unequal exact signatures, which fields repair the collisions,

Table 6: Projection-induced contract collisions and minimum repair fields for headline vocabularies.

Projection	Eq.	False	Cond.	Repair field(s)
Keyword	2,284	**254**	11.1%	**placement**
Yes/No	2,036	6	0.3%	layer
Operator-only	2,268	**238**	10.5%	**layer**

Table 7: Collision severity under lossy projections, measured by exact-signature distance and atom gaps.

Projection	False	Dist.	Mean Δ	Max Δ
Keyword	254	1.00	9.09	17
Yes/No	6	1.00	3.00	3
Operator	238	1.00	5.84	14
Placement	10,702	1.00	21.29	53
State	23,593	1.00	13.06	64
Surface	**0**	**0.00**	**0.00**	**0**

Table 8: Executable projection baselines. Bold zeros are exact or strengthened controls; underlining marks the worst one-field projection, exposing the shown range.

Baseline	Eq.	False	Rate
Exact	2,030	**0**	**0.0%**
Keyword	2,284	254	11.1%
Op/action	2,036	6	0.3%
Operator	2,268	238	10.5%
Kind-only	2,282	252	11.0%
Layer-only	2,479	449	18.1%
Place-only	12,732	10,702	84.1%
State-only	25,623	23,593	92.1%
Surface	2,030	**0**	**0.0%**

whether collisions reflect recognizable query-processing mechanisms rather than construction effects, and how much distinguishability each vocabulary retains. The schema, projections, and repair search are fixed before derived tables are generated: exact signatures define the reference partition; each coarse vocabulary declares candidate equivalence classes; repairs are evaluated over all counted witnesses. The table sequence is local to the argument: denominator checks precede projection measurements, robustness tables follow the main collision result, and repair tables appear where the certificate is introduced. The table column "false" is contract-level: it means that a declared vocabulary collapses unequal public optimizer-contract signatures, not that every witness yields a latency or plan-quality regression. The baselines in this section are comparison-denominator baselines, not competing optimizer implementations or semantic-cache systems; the fixed catalog grounds them as feature-name labels in engine docs, capability matrices in benchmark/workload descriptions, and EXPLAIN/operator-token comparisons in optimizer studies [12, 20, 21, 32, 33, 54, 55]. Exact, ablated, one-field, and strengthened projections are controls, not additional claims about deployed systems.

Coarse comparison induces collisions. Table 6 reports the main measurement. The keyword projection declares 2,284 equivalences: 2,030 pairs agree under exact public-contract signatures and 254 pairs disagree. The operator-only projection declares 2,268 equivalences, including 238 pairs whose exact signatures disagree. These errors are conditional on the baseline declaring equivalence, which is the decision faced by a user of a feature matrix. The results are stable under binomial and probe-cluster uncertainty estimates, and the collision count remains nonzero for keyword and operator-only projections after removing any single public source. A strict-projection negative control yields zero collisions.

Counts alone understate the collision. Table 7 measures how far apart the exact contracts are after a projection has declared them equal. For the headline projections, every collision pair has exact-signature Jaccard distance 1.0: the projected comparison has equated disjoint public contract signatures rather than near duplicates. Keyword collision pairs differ by 9.09 evidence atoms on average, and operator-only collision pairs differ by 5.84 atoms. The strengthened surface projection is the control: it returns to the exact-equivalence denominator and has zero projection-induced collision.

Table 8 is the vocabulary ablation. It evaluates exact negative controls, common feature matrices, one-field adversarial vocabularies, and strengthened high-information baselines. The exact-contract baseline produces no collisions. Keyword, kind-only, and operator-only comparisons induce collisions at similar rates, so the result is not tied to one synonym list. One-field baselines such as placement-only, decision-time-only, and state-only are much worse, showing that a single optimizer dimension is not a safe substitute for contract semantics. By contrast, the strengthened semantic-surface baseline has zero collisions, which localizes the missing fields rather than merely declaring all abstractions bad.

Projection kernels are measurable objects. Figure 5 reports the finite information profile of representative projections. Exact signatures form 1,288 observed classes. Keyword projection compresses them into 148 projected classes, 81 of which contain multiple exact classes, retaining 64% of empirical signature entropy and inflating declared equivalence by 12.5%. Operator-only projection retains more entropy but still leaves 191 colliding projected classes and 238 pairwise collisions. Placement-only and state-only baselines demonstrate the opposite failure mode: a single field creates very large kernels. The strengthened surface projection still has projected collisions at the signature-class level, but it does not produce any pairwise collision over the engine-probe comparison set. This is the empirical counterpart of the projection-kernel theorem: the kernel is finite, populated, and repairable.

Resolution-lattice stress test. The information profile asks how much a named projection compresses exact signatures. Table 9 asks a stricter question: if a comparison is allowed to retain any subset of public optimizer fields, which retained-field vocabularies are safe? The test enumerates all 256 subsets of the eight evidence-atom fields and evaluates them over the same 88,536 engine–probe comparisons. The empty vocabulary declares 59,546 unsupported equivalences. Every singleton semantic field is unsafe: operator alone leaves 238 collisions, kind alone leaves 252, layer alone leaves 449, and placement alone leaves 10,702. Excluding action-variant identifiers, the semantic field lattice has 128 subsets, 44 unsafe regions, and minimum safe field count two; the two minimum safe no-variant vocabularies are operator+layer and kind+placement.

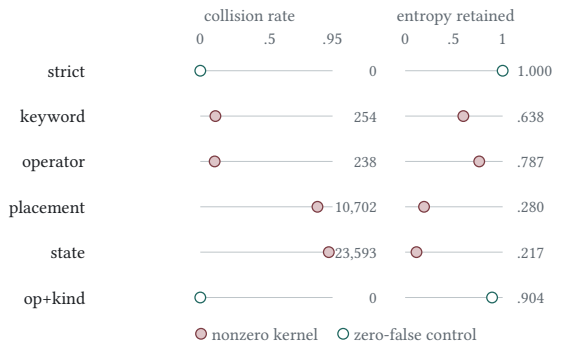

Figure 5: Projection information profile. Rows are projection vocabularies; axes show collision-rate and entropy-retention ratios, with false-pair counts beside nonzero kernels.

Table 9: Retained-field lattice for projection repair; false counts mark unsafe vocabularies.

Fields	$\|R\|$	Classes	Ent.	Eq.	False
None	0	1	-0.000	61,576	59,546
Operator	1	387	0.787	2,268	238
Action kind	1	183	0.661	2,282	252
Layer	1	52	0.552	2,479	449
Placement	1	9	0.280	12,732	10,702
Operator+layer	2	591	0.850	2,030	**0**
Kind+placement	2	214	0.680	2,030	**0**
Surface	5	820	0.904	2,030	**0**
All sem.	7	947	0.934	2,030	**0**

Table 10: Leave-one-source robustness of projection collisions across micro, pair-macro, and LOO views.

Proj.	Decl.	False	Micro	Pair	LOO
Keyword	**2284**	**254**	**0.111**	**0.402**	**[0.017,0.144]**
Yes/No	2036	6	0.003	0.003	[0.000,0.052]
Operator	**2268**	**238**	**0.105**	**0.290**	**[0.032,0.148]**

A compact set of ten concrete engine–probe witnesses covers all unsafe retained-field regions, so the frontier is not driven by one example row. Deriving the frontier antichains gives 14 minimal safe no-variant vocabularies and eight maximal unsafe vocabularies; the monotone frontier certificate shows that every safe retained-field vocabulary contains one of the minimal safe elements and every unsafe vocabulary is contained in a maximal unsafe element.

The robustness view in Table 10 checks that the headline result is not an effect of a single denominator. The micro rate conditions on the baseline declaring equivalence; the pair-macro rate averages within engine pairs; and the leave-one-source range removes all rules supported by one public source. Keyword and operator-only collisions remain visible across these views, with leave-one-source conditional-rate ranges $[0.017, 0.144]$ and $[0.032, 0.148]$ respectively.

The evidence side is cross-checked without retraining on the headline witnesses. Every collision witness has at least two public source records and side-balanced support on both compared

Table 11: Dispersion of collision witnesses across probes, engine pairs, and feature values.

Projection	Wit.	Probes	Pairs	Values	Max
Keyword	254	254	3	92	1
Operator-only	238	238	2	75	1
Yes/No	6	6	1	15	1

Table 12: Finite-comparison scalability and dispersion. False is witness count; Probes/Axes count distinct probes and feature axes; Fam. is pair-family regions with witnesses out of six; Checks/s is minimum throughput in the 8× lift; Drift is mismatch against full recomputation.

Proj.	False	Probes	Axes	Fam.	Checks/s	Drift
strict	0	–	–	0/6	264,891	0
keyword	254	254	9	3/6	327,059	0
operator-only	238	238	8	2/6	361,762	0
surface	0	–	–	0/6	282,181	0

signatures. Source leave-one-out changes the public-evidence denominator, feature-family holdout learns repairs away from one query slice, and engine-family stress changes the compared engine pairs; none creates an unresolved witness after the layer+placement repair. These checks are not meant to make a perfect repair surprising by itself; they show that the same small basis survives independent changes to source support, query-feature family, and engine-pair family after the schema is fixed.

The 287-rule corpus is a finite public-contract sample, not a census of optimizer internals, so the remaining checks ask whether the observed kernel is a one-source or one-shape accident. Keyword and operator-only collisions survive every leave-one-source removal; the 498 headline witnesses occupy 486 distinct probes; and the largest source affects 46 rules, 4,216 maps, and 14.3% of rule–probe rows. Table 11 further shows that witnesses touch every feature dimension and cover 92 and 75 observed feature values for keyword and operator-only projections. The witnesses are therefore dispersed projection-kernel evidence rather than duplicated rows.

Cost and scalability. Table 12 adds stress views. The finite audit executes all 88,536 engine–probe pair checks per projection, so the kernel is not sampled. The full audits complete in seconds with throughput above 50,000 comparisons per second; an 8× lift reaches 708,288 comparisons per projection with throughput above 200,000 comparisons per second and prefix-scaling $R^2 \geq 0.98$. Streaming maintenance has zero drift and uses 88,536 work units rather than the 186.7 million implied by recomputing every prefix. A source refresh is local: the largest public source affects 46 rules, 4,216 maps, and 14.3% of rule–probe map rows. The table also ties scaling to dispersion and family stratification: keyword collisions span 254 probes, nine feature axes, and three of six pair-family regions; operator-only spans 238 probes, eight axes, and two regions.

The probe-subsample view checks that the collision result is not an effect of using the full generated benchmark. At 10% and 50% probe subsamples, keyword and operator-only projections retain at least one collision in every trial, with median conditional rates

Table 13: Repair fields versus attribution leaders across headline projections.

Projection	Single-field repairs	Top attribution fields
Keyword	variant, **placement**	variant=.168; **placement=.168**
Yes/No	variant, layer, placement, state	each=.250
Operator-only	variant, **layer**	variant=.184; **layer=.184**

Table 14: Projection repair ladder showing residual false pairs after restoring fields.

Projection	Restored fields	Eq.	False
Keyword	–	2,284	254
Keyword	operator	2,036	6
Keyword	layer	2,072	42
Keyword	placement	2,030	**0**
Keyword	layer+placement	2,030	**0**
Yes/No	–	2,036	6
Yes/No	layer	2,030	**0**
Yes/No	placement	2,030	**0**
Operator	–	2,268	238
Operator	kind	2,036	6
Operator	placement	2,062	32
Operator	layer	2,030	**0**
Operator	layer+placement	2,030	**0**

close to the full benchmark. The weaker yes/no projection is sparse at 10%, but becomes nonzero in every trial by 50%.

The mutation-suite view is adversarial: it expands the comparison vocabulary around the headline baselines rather than only evaluating the three projections used in the abstract. Exact signatures remain a zero-false negative control, one-field projections produce much larger kernels, and strengthened projections that restore operator, action kind, and the public semantic surface collapse to the exact-equivalence denominator. The result is not that every abstraction is bad; it is that common optimizer-comparison abstractions omit query-processing fields that several independently weakened projections need and several strengthened projections restore.

Repair certificates identify the missing semantics. The projection calculus makes repair checkable, but the empirical question is which fields are sufficient. The repair column in Table 6 reports minimum-cardinality universal repairs and held-out-probe generalization. Placement repairs every keyword collision. Layer repairs every operator-only collision. The learned repairs resolve all held-out collisions across probe folds. A fixed semantic basis consisting of optimizer layer and execution placement repairs all observed collisions across keyword, yes/no, and operator-only projections, even when fine-grained action-variant labels are excluded. The certificate is checked as a finite proof obligation: each counted witness is full-signature unequal, projection-equal, separated by the reported repair, and protected by a grounded counter-witness against every smaller universal repair. The hitting-set view over all 498 headline witnesses agrees exactly with direct repaired-signature enumeration.

Table 15: Held-out feature-family repair stability for point minima and the fixed basis.

Projection	Folds	Held-out	Point	Basis
Keyword	12	830	144	**0**
Operator	9	569	12	**0**
Yes/No	1	3	**0**	**0**

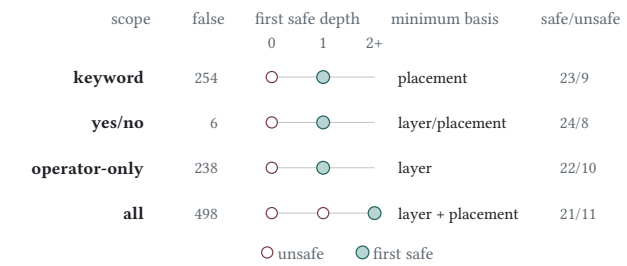

Figure 6: Semantic frontier over retained fields. Rows are projection scopes; the depth axis marks the first collision-free retained-field set, with layer+placement required for the joint scope.

Table 15 adds a stronger anti-overfitting test. Each fold withholds a non-default query-feature family, learns repairs from the remaining collision witnesses, and tests them on the held-out family. A point minimum can choose an underspecified field when the training denominator is narrow; the fixed layer and placement basis resolves every held-out collision across all folds. The repair is therefore not a post-hoc fit to one query slice, but a stable contract denominator.

The repair ladder in Table 14 gives a stepwise view of the same diagnosis. Restoring operator alone nearly repairs keyword projection but leaves six collisions; placement eliminates them. For operator-only projection, layer eliminates all collisions while placement alone leaves residual errors. Figure 6 then reports the finite field-lattice frontier: safe subsets repair every collision in scope, unsafe subsets retain grounded counter-witnesses, and the all-projection row requires layer+placement for all 498 headline witnesses.

Table 13 shows why the repair fields are query-processing fields. Placement separates local execution, source delegation, distributed workers, and cloud-service execution. Layer and decision time separate logical rewrite, physical planning, distributed planning, and runtime adaptation. Operator and observability keep plan operators, plan evidence, physical estimates, and runtime profiles from being treated as interchangeable. The highest-attribution fields are the same fields that repair the collisions, and a fixed layer+placement semantic basis resolves every observed engine-pair hotspot without relying on fine-grained variant identifiers.

Collisions have mechanisms and hotspots. Table 16 groups collision witnesses into optimizer mechanisms and lists the largest engine-pair hotspots. The dominant mechanisms are action-variant, execution-placement, optimizer-layer, plan-evidence, and decision-time collisions. The hotspots are not random; they occur where public contracts expose different query-processing surfaces, such

Table 16: Collision mechanisms and representative engine-pair hotspots.

Mechanism	W.	Hotspot	False
Action variant	498	Keyword BQ–CH	203
Placement	466	Keyword BQ–Trino	45
Optimizer layer	430	Keyword CH–Trino	6
Plan evidence	366	Yes/No CH–Trino	6
Decision time	362	Operator CH–Trino	123
Operator family	281	Operator Spark–Trino	115

Table 17: Workload-feature contract movements under controlled toggles.

Engine	Feature	Mean dist.	Nonzero
DuckDB	reuse_structure	**0.827**	0.854
Trino	statistics_need	0.742	0.853
DuckDB	source_boundary	0.590	0.623
Trino	distribution_trigger	0.519	0.861
Spark SQL	adaptivity_trigger	0.442	0.858
BigQuery	distribution_trigger	0.373	0.891
Spark SQL	join_type	0.369	0.810
ClickHouse	order_limit	0.351	0.511

as source delegation versus local planning, runtime adaptivity versus compile-time planning, and stage-graph observability versus estimated local plans.

Optimizer contracts are workload-sensitive. Table 17 reports controlled feature-toggle movements. The largest movements involve CTE reuse, source boundary, statistics need, distribution triggers, and adaptivity triggers. Optimizer-contract comparability is therefore not an engine-level scalar; it is a relation between engine contracts and query structure.

The evaluation links theory and evidence: the nontrivial kernel is populated by public optimizer evidence rather than synthetic examples. The information profile, frontier certificate, and SQL validation show that the denominator is distinguishable, repairable, and executable on deterministic catalogs plus DuckDB and PostgreSQL.

6 GROUNDED CASE STUDIES

Table 18 gives the concrete rule pairs behind the aggregate measurements. The rows are selected to span the major mechanism families in Table 16, not because they are unusually easy repairs. C1 is the expanded example: a remote-source aggregate probe selects category and sum(amount), filters by region, and groups by category. BIGQUERY-FEDERATED contributes a source-delegated filter contract plus a CloudSQL blocking condition; TRINO-PUSH contributes aggregate pushdown, aggregate-expression blocking, and join-pushdown contracts. Under keyword projection, both maps contain "pushdown," so $S_\pi(M_{\mathrm{BQ}}) = S_\pi(M_{\mathrm{Trino}})$. Under exact signatures, filter delegation with stage evidence differs from aggregate delegation plus aggregate-expression denial. The certificate checks exact inequality, projection equality, separation after retaining placement and operator/action variant, and a smaller-field counter-witness.

C2 separates PostgreSQL compile-time estimates from BigQuery stages, shuffle metrics, profile-time evidence, and Snowflake logical plans. C3 separates compile-time choices from runtime adaptation: BigQuery runtime repartition, Spark AQE, and ClickHouse automatic join conversion are not the same contract as static join selection [26, 52, 53, 63, 73, 74]. C4 uses CTE reuse probes to contrast DuckDB inline/materialize/reuse guards from DUCK-WITH with PostgreSQL WITH/materialization controls. In an engine-selection or migration report, these certificates change the conclusion from "both systems support the feature" to a checkable statement about which layer, placement, and evidence surface must be preserved before performance numbers are compared. These worked cases are representative, not cherry-picked exceptions: the same rule schema and finite certificate checker cover all 498 headline witnesses.

7 IMPLICATIONS AND RELATED WORK

Optimizer interfaces as semantic interfaces. A public optimizer claim should name the compared action, guard, layer, placement, decision time, and evidence surface. These fields do not expose proprietary cost formulas; they state whether heterogeneous engines, adapters, remote sources, and service-side stages are being compared on the same query-processing phenomenon [9, 25, 28, 29, 32].

Relation to prior work. Classical optimizers and rule frameworks separate equivalence, access paths, search, costing, and adaptation [5, 11, 35–37, 47, 66, 68]; OptSem-C asks which distinctions are public enough for cross-engine comparison. Calcite traits such as convention, collation, and distribution are internal annotations for rule matching and planning; OptSem-C uses such traits only when documented or exposed, and adds modality, guard state, decision time, observability, and source support so that claims about different engines can be audited outside one optimizer's trait system. SQL-equivalence and database-testing systems decide result preservation or detect wrong-answer bugs [1, 10, 15, 16, 65, 71]; OptSem-C instead checks the comparison object before a performance or correctness experiment begins. Exact-cardinality testing, JOB, adaptive processing, learned optimizers, and workload benchmarks sharpen denominators when older metrics hide the behavior of interest [3, 4, 6, 12, 14, 20, 21, 23, 24, 27, 44, 50, 54–58, 62, 70, 76].

Protocol and scope. Every optimizer comparison is a projection. OptSem-C is therefore a finite-certificate workflow rather than a latency predictor: choose a vocabulary, extract public contracts, validate the executable probe denominator, compute the projection kernel, and publish either retained fields or unresolved assumptions. The formal theorems make this workflow checkable; the accompanying materials instantiate it on public database-engine evidence.

Coverage and limits. The unit of evidence is a public optimizer contract, not inferred private state. A rule absent from manuals, plans, diagnostics, connector behavior, or observable SQL behavior is outside the comparison unless it changes the public contract. The repair certificates are complete for the finite grounded corpus, but they do not prove that no future engine or undocumented feature needs another field. Published-motif crosswalks, source leave-one-out tests, feature holdouts, engine-family stress, and real-engine SQL checks mitigate circularity by changing the denominator, source support, or execution surface; they do not turn public contracts into private ground truth.

Table 18: Worked public-contract cases linking coarse labels to hidden contract pairs and repair fields.

Case	Coarse label	Contract pair hidden by the label	Separating fields	Risk closed
C1	pushdown=yes	BigQuery federated filter delegation vs. Trino aggregate/expression pushdown	**operator, variant, placement**	delegation is not local rewrite
C2	plan evidence	PostgreSQL EXPLAIN estimates vs. BigQuery stage/shuffle evidence and Snowflake logical plans	**observability, placement**	plan surface is not execution semantics
C3	adaptive=yes	Compile-time planning vs. BigQuery/Spark/ClickHouse runtime adaptation	**layer, decision time**	runtime adaptation is not compile-time planning
C4	CTE support	DuckDB/PostgreSQL CTE inline, materialize, reuse, and disable guards	**variant, guard state**	feature support depends on guard obligations

Using repair certificates. A repair certificate changes how comparisons are stated. Instead of saying that two systems both "support pushdown" or both "use adaptive optimization," a study can state the certified frontier: local execution versus source delegation, logical rewrite versus physical planning, compile time versus runtime, and plan evidence versus contract evidence. The certificate exposes retained fields, collapsed exact signatures, and counter-witnesses before a performance explanation is accepted. OptSem-C does not impose one taxonomy; it exposes when source delegation, local execution, runtime adaptation, or plan evidence are treated as interchangeable.

Reproducibility and replay. The reproducibility materials contain the contract corpus, evidence metadata, generated SQL probes, projection-kernel calculators, repair-certificate scripts, DuckDB and PostgreSQL validation logs, table and figure data, and the claim-evidence graph. Replay rebuilds derived outputs, regenerates the SQL probe set, reruns deterministic catalog checks, reruns DuckDB and PostgreSQL checks when those engines are installed, recompiles the manuscript, and reruns integrity suites; archival deposit is the next reproducibility step rather than hidden evidence.

Validity threats. Public behavior and documentation can drift. OptSem-C records version scope and evidence digests, treats a source refresh as an incremental audit, and separates documentation claims from exposed-behavior checks. Commercial systems and emerging learned operators may require additional action variants or fields.

Contract-reality alignment. The alignment claim is layered: public commitment records a documented optimizer action or guard; public observability records a plan, profile, stage graph, or diagnostic surface; executability requires generated probes to be parseable, plannable, executable, and shape-consistent. A successful SQL run proves that the denominator is executable, not that all documented contracts fired in a runtime trace.

Interpreting published-motif coverage. The motif suite is not a benchmark leaderboard or a reproduction of TPC-H, TPC-DS, JOB, SSB, or data-lake workloads. It asks whether the generated feature space contains optimizer-decision requirements already used to study join order, cardinality estimation, adaptive processing, star schemas, cross-source analytics, and optimizer controls. Coverage plus motif executability does not prove that all future workload families are represented, but it prevents the denominator from being justified only by the same rules used to count collisions.

Why the minimum fields matter. The layer+placement basis is not a universal taxonomy; it is a minimum safe basis for the observed headline projections and finite grounded corpus. Placement distinguishes local execution, source delegation, distributed workers, and service-side execution. Layer distinguishes rewrite, physical planning, distributed planning, runtime adaptation, and plan evidence. Future engines may require more fields, but stronger vocabularies should preserve these distinctions or explain why their target comparison does not need them.

Boundary. OptSem-C does not choose the best physical plan, validate private cloud rules, or measure learned-cost improvements. It exposes the comparison boundary before those studies: if a vocabulary collapses source delegation with local execution or runtime adaptation with compile-time planning, the later performance explanation already carries an unstated semantic assumption.

8 CONCLUSION

A coarse optimizer comparison can declare equivalence where public contracts disagree. OptSem-C grounds that failure mode in public evidence, turns comparison vocabularies into finite projections, and measures the resulting projection kernel over 287 rules and 4,216 probes. In this finite corpus, layer+placement separates all 498 headline witnesses, and every unsafe retained-field region has a concrete counter-witness. Optimizer studies should publish the comparison vocabulary and a repair certificate alongside performance results.

The broader lesson is methodological. A comparison denominator is part of the experimental claim, not a caption for the plot that follows. Before a study ranks two engines, ports a workload, or explains a performance delta, it should state which optimizer actions are being equated, which public evidence supports the equality, which fields have been projected away, and whether any observed counter-witness remains. OptSem-C supplies that report in finite form: the exact signatures define the reference relation, the projection kernel lists the assumptions introduced by the chosen vocabulary, and the repair frontier states the smallest public distinctions needed to make the comparison safe for the declared scope.

This framing does not replace conventional query-processing experiments. It makes them easier to interpret. A latency benchmark, optimizer-regression suite, or migration study can still measure plans, runtime, cost estimates, and user-visible outcomes; OptSem-C checks that the semantic object being compared is stable enough for those measurements to support the intended conclusion. When the kernel is empty, the study can say so. When it is nonempty, the study can either retain the missing fields or explicitly publish the unresolved assumption. That discipline turns a feature checklist into an auditable contract between evidence, vocabulary, and empirical claim.

REFERENCES

[1] A. V. Aho, Y. Sagiv, and J. D. Ullman. 1979. Equivalences among Relational Expressions. *SIAM J. Comput.* (1979). https://doi.org/10.1137/0208017

[2] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. *SIGMOD* (2015). https://doi.org/10.1145/2723372.2742797

[3] T. G. Armstrong, V. Ponnekanti, D. Borthakur, and M. Callaghan. 2013. LinkBench: A Database Benchmark Based on the Facebook Social Graph. *SIGMOD* (2013). https://doi.org/10.1145/2463676.2465296

[4] R. Avnur and J. M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. *SIGMOD* (2000). https://doi.org/10.1145/342009.335420

[5] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. *SIGMOD* (2018). https://doi.org/10.1145/3183713.3190662

[6] D. Bitton, D. J. DeWitt, and C. Turbyfill. 1983. Benchmarking Database Systems: A Systematic Approach. *VLDB* (1983). https://www.vldb.org/conf/1983/P008.PDF

[7] P. A. Boncz, M. Zukowski, and N. Nes. 2005. MonetDB/X100: Hyper-Pipelining Query Execution. *CIDR* (2005). https://www.cidrdb.org/cidr2005/papers/P19.pdf

[8] P. Buneman, S. Khanna, and W.-C. Tan. 2001. Why and Where: A Characterization of Data Provenance. *ICDT* (2001). https://doi.org/10.1007/3-540-44503-X_20

[9] Michael J. Carey, Laura M. Haas, Peter M. Schwarz, Manish Arya, William F. Cody, Ronald Fagin, Myron Flickner, Allen W. Luniewski, Wayne Niblack, Dragutin Petkovic, John Thomas, John H. Williams, and Edward L. Wimmers. 1995. Towards Heterogeneous Multimedia Information Systems: The Garlic Approach. *RIDE-DOM* (1995). https://research.ibm.com/publications/towards-heterogeneous-multimedia-information-systems-the-garlic-approach

[10] A. K. Chandra and P. M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. *STOC* (1977). https://doi.org/10.1145/800105.803397

[11] S. Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. *PODS* (1998). https://doi.org/10.1145/275487.275492

[12] S. Chaudhuri, V. R. Narasayya, and R. Ramamurthy. 2009. Exact Cardinality Query Optimization for Optimizer Testing. *PVLDB* (2009). https://doi.org/10.14778/1687627.1687739

[13] J. Cheney, L. Chiticariu, and W.-C. Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* (2009). https://doi.org/10.1561/1900000006

[14] Y. Chronis, Y. Wang, Y. Gan, S. Abu-El-Haija, C. Lin, C. Binnig, and F. Ozcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. arXiv. https://arxiv.org/abs/2408.16170

[15] S. Chu, B. Murphy, J. Roesch, A. Cheung, and D. Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *PVLDB* (2018). https://doi.org/10.14778/3236187.3236200

[16] S. Chu, C. Wang, K. Weitz, and A. Cheung. 2017. Cosette: An Automated Prover for SQL. *CIDR* (2017). https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf

[17] Google Cloud. 2026. BigQuery Documentation: Query Plan Explanation. Documentation. https://cloud.google.com/bigquery/docs/query-plan-explanation

[18] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* (1970). https://doi.org/10.1145/362384.362685

[19] R. L. Cole and G. Graefe. 1994. Optimization of Dynamic Query Evaluation Plans. *SIGMOD* (1994). https://doi.org/10.1145/191839.191872

[20] Transaction Processing Performance Council. 2024. TPC Benchmark DS Standard Specification. TPC. https://www.tpc.org/tpcds/

[21] Transaction Processing Performance Council. 2024. TPC Benchmark H Standard Specification. TPC. https://www.tpc.org/tpch/

[22] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. *SIGMOD* (2016). https://doi.org/10.1145/2882903.2903741

[23] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *PVLDB* (2024). https://doi.org/10.14778/3659437.3659448

[24] A. Deshpande, Z. Ives, and V. Raman. 2007. Adaptive Query Processing. *Foundations and Trends in Databases* (2007). https://doi.org/10.1561/1900000001

[25] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. 2015. The BigDAWG Polystore System. *SIGMOD Record* (2015). https://doi.org/10.1145/2814710.2814713

[26] A. Dutt, C. Wang, A. Nazi, S. Kandula, V. Narasayya, and S. Chaudhuri. 2019. Selectivity Estimation for Range Predicates Using Lightweight Models. *PVLDB* (2019). https://doi.org/10.14778/3329772.3329780

[27] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC Social Network Benchmark: Interactive Workload. *SIGMOD* (2015). https://doi.org/10.1145/2723372.2742786

[28] Apache Software Foundation. 2026. Apache Calcite Documentation: Adapter and Optimizer Framework. Documentation. https://calcite.apache.org/docs/

[29] Apache Software Foundation. 2026. Apache DataFusion Documentation: Query Optimizer. Documentation. https://datafusion.apache.org/user-guide/explain-usage.html

[30] Apache Software Foundation. 2026. Apache Spark SQL Performance Tuning. Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[31] DuckDB Foundation. 2026. DuckDB Documentation: WITH Clause and CTE Materialization. Documentation. https://duckdb.org/docs/stable/sql/query_syntax/with.html

[32] Presto Foundation. 2026. Presto Documentation: Cost-Based Optimizer. Documentation. https://prestodb.io/docs/current/optimizer/cost-based-optimizations.html

[33] Trino Software Foundation. 2026. Trino Documentation: Cost-Based Optimizations and Pushdown. Documentation. https://trino.io/docs/current/optimizer/cost-based-optimizations.html

[34] Cesar A. Galindo-Legaria and Arnon Rosenthal. 1997. Outerjoin Simplification and Reordering for Query Optimization. *ACM Transactions on Database Systems* (1997). https://doi.org/10.1145/244810.244812

[35] G. Graefe. 1990. Encapsulation of Parallelism in the Volcano Query Processing System. *SIGMOD* (1990). https://doi.org/10.1145/93597.98720

[36] G. Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Engineering Bulletin* (1995). https://www.sigmod.org/publications/dblp/db/journals/debu/Graefe95a.html

[37] G. Graefe and W. J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. *ICDE* (1993). https://doi.org/10.1109/ICDE.1993.344061

[38] T. J. Green. 2011. Containment of Conjunctive Queries on Annotated Relations. *Theory of Computing Systems* (2011). https://doi.org/10.1007/s00224-011-9327-6

[39] T. J. Green, G. Karvounarakis, and V. Tannen. 2007. Provenance Semirings. *PODS* (2007). https://doi.org/10.1145/1265530.1265535

[40] PostgreSQL Global Development Group. 2026. PostgreSQL Documentation: Using EXPLAIN and Query Planning. Documentation. https://www.postgresql.org/docs/current/using-explain.html

[41] Anurag Gupta, Deepak Agarwal, Derek Tan, Jakub Kulesza, Rahul Pathak, Stefano Stefani, and Vidhya Srinivasan. 2015. Amazon Redshift and the Case for Simpler Data Warehouses. *SIGMOD* (2015). https://doi.org/10.1145/2723372.2742795

[42] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *PVLDB* (2021). https://doi.org/10.14778/3503585.3503586

[43] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, and C. Binnig. 2020. DeepDB: Learn from Data, Not from Queries! *PVLDB* (2020). https://doi.org/10.14778/3384345.3384349

[44] ClickHouse Inc. 2024. ClickBench: A Benchmark for Analytical DBMS. Online benchmark. https://benchmark.clickhouse.com/

[45] ClickHouse Inc. 2026. ClickHouse Documentation: JOIN Optimization and Algorithms. Documentation. https://clickhouse.com/docs/guides/joining-tables

[46] Snowflake Inc. 2026. Snowflake Documentation: EXPLAIN. Documentation. https://docs.snowflake.com/en/sql-reference/sql/explain

[47] Y. E. Ioannidis. 1996. Query Optimization. *Comput. Surveys* (1996). https://doi.org/10.1145/234313.234367

[48] Z. G. Ives, D. Florescu, M. Friedman, A. Levy, and D. S. Weld. 1999. An Adaptive Query Execution System for Data Integration. *SIGMOD* (1999). https://doi.org/10.1145/304181.304209

[49] David Justen, Daniel Ritter, Campbell Fraser, Andrew Lamb, Allison Lee, Thomas Bodner, Mhd Yamen Haddad, Steffen Zeuch, Volker Markl, and Matthias Boehm. 2024. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance. *PVLDB* (2024). https://doi.org/10.14778/3648160.3648175

[50] N. Kabra and D. J. DeWitt. 1998. Efficient Mid-Query Re-Optimization of Sub-Optimal Query Execution Plans. *SIGMOD* (1998). https://doi.org/10.1145/276304.276315

[51] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. *CIDR* (2019). https://www.cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf

[52] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. 2018. The Case for Learned Index Structures. *SIGMOD* (2018). https://doi.org/10.1145/3183713.3196909

[53] Sanjay Krishnan, Zongheng Yang, Ken Goldberg, Joseph M. Hellerstein, and Ion Stoica. 2018. Learning to Optimize Join Queries With Deep Reinforcement Learning. arXiv. https://arxiv.org/abs/1808.03196

[54] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2015. How Good Are Query Optimizers, Really? *PVLDB* (2015). https://doi.org/10.14778/2850583.2850594

[55] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *VLDB Journal* (2018). https://doi.org/10.1007/s00778-017-0480-7

[56] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. *SIGMOD* (2021). https://doi.org/10.1145/3448016.3452838

[57] R. Marcus and O. Papaemmanouil. 2019. Neo: A Learned Query Optimizer. *PVLDB* (2019). https://doi.org/10.14778/3342263.3342644

[58] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdzic. 2004. Robust Query Processing Through Progressive Optimization. *SIGMOD* (2004). https://doi.org/10.1145/1007568.1007642

[59] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2010. Dremel: Interactive Analysis of Web-Scale Datasets. *PVLDB* (2010). https://doi.org/10.14778/1920841.1920886

[60] G. Moerkotte and T. Neumann. 2006. Analysis of Two Existing and One New Dynamic Programming Algorithm for the Generation of Optimal Bushy Join Trees without Cross Products. *VLDB* (2006). https://dblp.org/rec/conf/vldb/MoerkotteN06.html

[61] T. Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *PVLDB* (2011). https://doi.org/10.14778/2002938.2002940

[62] P. ONeil, E. ONeil, X. Chen, and S. Revilak. 2009. The Star Schema Benchmark and Augmented Fact Table Indexing. *TPCTC* (2009). https://www.cs.umb.edu/~poneil/StarSchemaB.PDF

[63] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. *CIDR* (2017). https://db.cs.cmu.edu/papers/2017/p42-pavlo-cidr17.pdf

[64] M. Raasveldt and H. Muehleisen. 2019. DuckDB: An Embeddable Analytical Database. *SIGMOD* (2019). https://doi.org/10.1145/3299869.3320212

[65] M. Rigger and Z. Su. 2020. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *ESEC/FSE*. 1140–1152. https://doi.org/10.1145/3368089.3409710

[66] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. *SIGMOD* (1979). https://doi.org/10.1145/582095.582099

[67] Jeff Shute, Radek Vingralek, Bart Samwel, Ben Handy, Chad Whipkey, Eric Rollins, Mircea Oancea, Kyle Littlefield, David Menestrina, Stephan Ellner, John Cieslewicz, Ian Rae, Traian Stancescu, and Himani Apte. 2013. F1: A Distributed SQL Database That Scales. *PVLDB* (2013). https://doi.org/10.14778/2536222.2536232

[68] Mohamed A. Soliman, Lyublena Antova, Venkatesh Raghavan, Amr El-Helw, Zhongxian Gu, Entong Shen, George C. Caragea, Carlos Garcia-Alvarado, Foyzur Rahman, Michalis Petropoulos, Florian Waas, Sivaramakrishnan Narayanan, Konstantinos Krikellas, and Rhonda Baldwin. 2014. Orca: A Modular Query Optimizer Architecture for Big Data. *SIGMOD* (2014). https://doi.org/10.1145/2588555.2595637

[69] M. Steinbrunn, G. Moerkotte, and A. Kemper. 1997. Heuristic and Randomized Optimization for the Join Ordering Problem. *VLDB Journal* (1997). https://doi.org/10.1007/s007780050040

[70] C. Turbyfill, C. Orji, and D. Bitton. 1993. AS3AP: An ANSI SQL Standard Scaleable and Portable Benchmark for Relational Database Systems. *The Benchmark Handbook* (1993). https://www.odbms.org/2014/03/benchmark-handbook-1993/

[71] S. Wang, S. Pan, and A. Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *PVLDB* (2024). https://doi.org/10.14778/3681954.3682024

[72] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *SIGMOD* (2023). https://arxiv.org/abs/2212.05526

[73] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. *SIGMOD* (2022). https://doi.org/10.1145/3514221.3517885

[74] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica. 2019. Deep Unsupervised Cardinality Estimation. *PVLDB* (2019). https://doi.org/10.14778/3368289.3368294

[75] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *NSDI* (2012). https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia

[76] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, and J. Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *PVLDB* (2023). https://doi.org/10.14778/3583140.3583160